

WiFi-Powered Robot Final Report

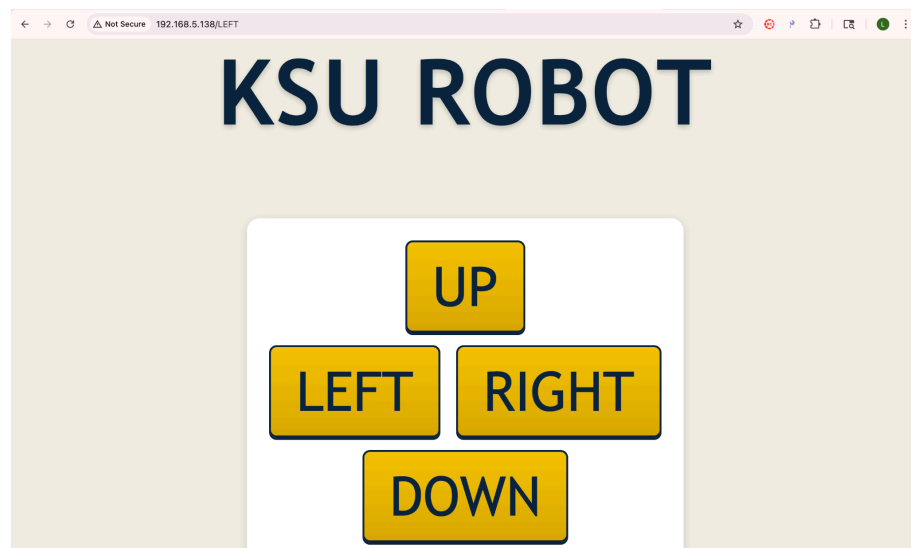
Lucy Hennessy and TJ Laryea

Project Overview and Introduction

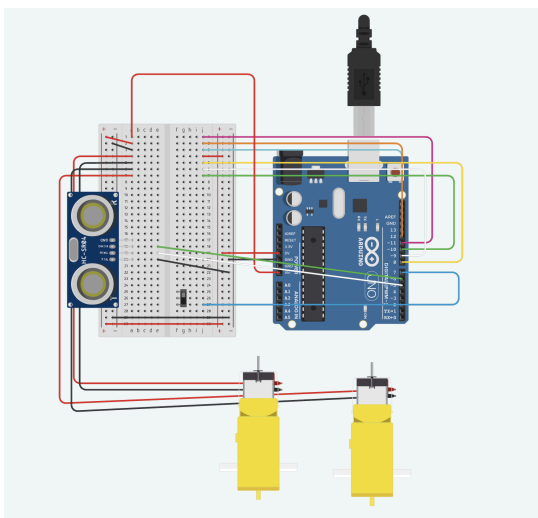
For this project, we were tasked with creating a wifi controlled robot with the Arduino R4 board that takes inputs from a web browser. The robot would use the Ultrasonic Sensor and it should be able to slow down and speed up based on the distance it is from an object.

Design Process

1. Web Interface

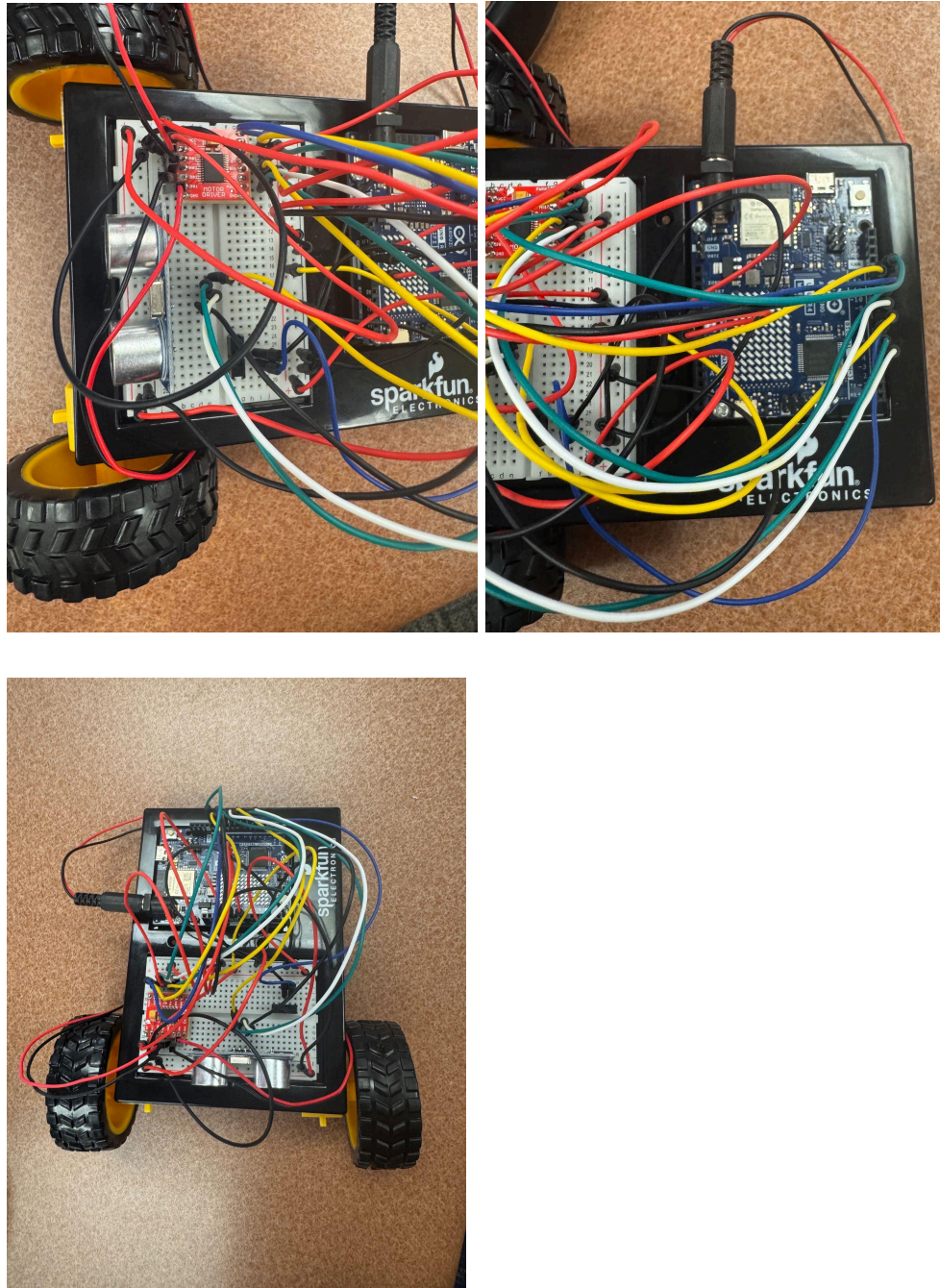


2. TinkerCAD



We couldn't find a piece to act as the motor driver but the wires are all wired as if it is there.

3. Wiring



Code

/*

```

WiFi Web Server Robot Control (UNO R4 WiFi)

Opens a simple web page with Up/Down/Left/Right buttons.
Robot:
- Controlled over WiFi
- Uses ultrasonic sensor to detect obstacles
- Stops and shows a circle on the LED matrix if obstacle < 15 cm
- Does NOT resume automatically (waits for next button press)
*/

#include "WiFiS3.h"
#include <Arduino_LED_Matrix.h>

// Motor driver pins, sending signals to the motors
const int AIN1 = 13;
const int AIN2 = 12;
const int PWMA = 11;

const int PWMB = 10;
const int BIN2 = 9;
const int BIN1 = 8;

// On/off switch for the robot
const int switchPin = 7;

//Ultrasonic sensor pins, how the robot measures distance
const int trigPin = 6;
const int echoPin = 5;

//Distance and blocking for the sensor
float distance = 0;
bool blocked = false;

// Code to have the robot automatically resume movement, says the last motor speeds
int lastRight = 0;
int lastLeft = 0;

// Sets the same speed everywhere
int baseSpeed = 200;

// LED matrix object, used for showing the circle
ArduinoLEDMatrix matrix;

// WiFi Connection
char ssid[] = "Scorcher";
char pass[] = "12345678";
int keyIndex = 0;

int status = WL_IDLE_STATUS;
WiFiServer server(80); // opens the actual web server

//All of the functions
void rightMotor(int motorSpeed);
void leftMotor(int motorSpeed);
void stopMotors();
float readDistance();
void showCircle();
void clearMatrix();
void printWifiStatus();
void resumeMotors();

//Movement commands
void moveUp() {

```

```

Serial.println("Command: UP");
if (blocked) return; // robot will not move when it is blocked
//saying whatever speed it is using right now
lastRight = baseSpeed;
lastLeft = -baseSpeed;
//actually moving the motors
rightMotor(lastRight);
leftMotor(lastLeft);
}

void moveDown() {
  Serial.println("Command: DOWN");
  if (blocked) return;
  lastRight = -baseSpeed;
  lastLeft = baseSpeed;
  rightMotor(lastRight);
  leftMotor(lastLeft);
}

void moveLeft() {
  Serial.println("Command: LEFT");
  if (blocked) return;
  lastRight = baseSpeed; // right wheel forward
  lastLeft = -baseSpeed; // left wheel backward
  rightMotor(lastRight);
  leftMotor(lastLeft);
}

void moveRight() {
  Serial.println("Command: RIGHT");
  if (blocked) return;
  lastRight = baseSpeed / 2; //turing right is slightly diferent speeds to make it
smoother
  lastLeft = -baseSpeed;
  rightMotor(lastRight);
  leftMotor(lastLeft);
}

//Automatically resuming movement in whatever direction it was already moving
void resumeMotors() {
  Serial.println("Auto-resuming previous motion...");
  rightMotor(lastRight);
  leftMotor(lastLeft);
}

// Setup runs once the robot turns on
void setup() {
  Serial.begin(9600);
  pinMode(switchPin, INPUT_PULLUP);

  // Motor pins sending signals to the driver
  pinMode(AIN1, OUTPUT);
  pinMode(AIN2, OUTPUT);
  pinMode(PWMA, OUTPUT);

  pinMode(BIN1, OUTPUT);
  pinMode(BIN2, OUTPUT);
  pinMode(PWMB, OUTPUT);

  // Ultrasonic pins setup
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  // start the LED matrix and clear anything already on it

```

```

matrix.begin();
clearMatrix();

// WiFi setup
if (WiFi.status() == WL_NO_MODULE) {
  Serial.println("Communication with WiFi module failed!");
  while (true)
    ;
}

// Make sure the firmware is updated
String fv = WiFi.firmwareVersion();
if (fv < WIFI_FIRMWARE_LATEST_VERSION) {
  Serial.println("Please upgrade the firmware");
}

// Keep trying to connect to wifi until it is successful
while (status != WL_CONNECTED) {
  Serial.print("Attempting to connect to Network: ");
  Serial.println(ssid);

  status = WiFi.begin(ssid, pass);
  delay(10000); // gives the board time to connect
}

server.begin();
printWifiStatus();
}

//loop for distance reading
void loop() {

  // Continuous obstacle checking
  distance = readDistance();

  //if the sensor fails return, just ignore it
  if (distance <= 0) {
    //under 15cm it stops immediately
  } else if (distance < 15) {
    if (!blocked) {
      blocked = true;
      stopMotors();
      showCircle(); // shows the warning circle
      Serial.println("Obstacle detected → STOP");
    }
    //Past 20cm, it is safe again so resume
  } else if (distance > 20) {
    if (blocked) {
      blocked = false;
      clearMatrix();
      Serial.println("Obstacle cleared → RESUMING");
      resumeMotors();
    }
  }
}

// Checking for someone connecting to the webpage
WiFiClient client = server.available();

if (client) {
  Serial.println("new client");
  String currentLine = "";

  while (client.connected()) {
    if (client.available()) {

```

```

char c = client.read();
Serial.write(c);

//When we hit a blank line, that means the browser is done sending headers
if (c == '\n') {
    if (currentLine.length() == 0) {

        //HTML and CSS for the webpage
        client.println("HTTP/1.1 200 OK");
        client.println("Content-Type: text/html; charset=UTF-8");
        client.println("Connection: close");
        client.println();

        client.println("<!DOCTYPE html>");
        client.println("<html>");
        client.println("<head>");
        client.println("<meta name=\"viewport\" content=\"width=device-width,
initial-scale=1\">");
        client.println("<title> Robot Control</title>");
        // styling for the entire webpage
        client.println("<style>");
        client.println("body {");
        client.println("    font-family: 'Trebuchet MS', Arial, sans-serif;");
        client.println("    text-align: center;");
        client.println("    background: #f2efe3;");
        client.println("    color: #0c2340;");
        client.println("    margin: 0;");
        client.println("}");

        // styling for the title
        client.println("h1 {");
        client.println("    font-size: 10vw;");
        client.println("    margin-top: 20px;");
        client.println("    letter-spacing: 2px;");
        client.println("    font-weight: 900;");
        client.println("    text-shadow: 0px 4px 8px rgba(0,0,0,0.2);");
        client.println("}");

        //outer box holding the buttons
        client.println(".panel {");
        client.println("    background: #ffffff;");
        client.println("    border-radius: 20px;");
        client.println("    padding: 25px;");
        client.println("    margin: 20px;");
        client.println("    box-shadow: 0 4px 15px rgba(0,0,0,0.15);");
        client.println("    display: inline-block;");
        client.println("}");

        //coding for the different buttons on the webpage
        client.println(".btn {");
        client.println("    font-family: 'Trebuchet MS', Arial, sans-serif;");
        client.println("    font-size: 6vw;");
        client.println("    padding: 20px 40px;");
        client.println("    margin: 10px;");
        client.println("    border: 3px solid #0c2340;");
        client.println("    border-radius: 12px;");
        client.println("    background: linear-gradient(#f7c400, #d8a900);");
        client.println("    color: #0c2340;");
        client.println("    box-shadow: 0px 4px 0px #0c2340;");
        client.println("    cursor: pointer;");
        client.println("    transition: 0.15s;");
        client.println("}");

```



```

        //button clicking animation
        client.println(".btn:active {}");
        client.println("  box-shadow: 0px 2px 0px #0c2340;");
        client.println("  transform: translateY(2px);");
        client.println("{}");

        //hover effect
        client.println(".btn:hover {}");
        client.println("  filter: brightness(1.15);");
        client.println("{}");
        client.println("</style>");

        client.println("</head>");
        client.println("<body>");

        client.println("<h1>ROBOT CONTROLS</h1>");

        client.println("<div class='panel'>");

        //Up
        client.println("<div><a href='/UP'><button
class='btn'>UP</button></a></div>");

        //Left and right
        client.println("<div>");
        client.println("<a href='/LEFT'><button class='btn'>LEFT</button></a>");
        client.println("<a href='/RIGHT'><button class='btn'>RIGHT</button></a>");
        client.println("</div>");

        //down
        client.println("<div><a href='/DOWN'><button
class='btn'>DOWN</button></a></div>");

        client.println("</div>");

        client.println("</body></html>");
        client.println();

        break;

    } else {
        currentLine = "";
    }
} else if (c != '\r') {
    currentLine += c;
}

//checking which button the user pressed and run that movement command
if (currentLine.indexOf("GET /UP") >= 0) moveUp();
if (currentLine.indexOf("GET /DOWN") >= 0) moveDown();
if (currentLine.indexOf("GET /LEFT") >= 0) moveLeft();
if (currentLine.indexOf("GET /RIGHT") >= 0) moveRight();
}
}
client.stop();
Serial.println("client disconnected");
}
}

//WiFi Status
void printWifiStatus() {

```



```

Serial.print("SSID: ");
Serial.println(WiFi.SSID());

IPAddress ip = WiFi.localIP();
Serial.print("IP Address: ");
Serial.println(ip);

long rssi = WiFi.RSSI();
Serial.print("RSSI: ");
Serial.print(rssi);
Serial.println(" dBm");

Serial.print("Open browser to: http://");
Serial.println(ip);
}

//Motor controls
void rightMotor(int motorSpeed) {
  motorSpeed = constrain(motorSpeed, -255, 255); //keeps speed in a range

  if (motorSpeed > 0) {
    digitalWrite(AIN1, HIGH);
    digitalWrite(AIN2, LOW);
    analogWrite(PWMA, motorSpeed);
  } else if (motorSpeed < 0) {
    digitalWrite(AIN1, LOW);
    digitalWrite(AIN2, HIGH);
    analogWrite(PWMA, -motorSpeed);
  } else {
    analogWrite(PWMA, 0); // stop
  }
}

void leftMotor(int motorSpeed) {
  motorSpeed = constrain(motorSpeed, -255, 255);

  if (motorSpeed > 0) {
    digitalWrite(BIN1, HIGH);
    digitalWrite(BIN2, LOW);
    analogWrite(PWMB, motorSpeed);
  } else if (motorSpeed < 0) {
    digitalWrite(BIN1, LOW);
    digitalWrite(BIN2, HIGH);
    analogWrite(PWMB, -motorSpeed);
  } else {
    analogWrite(PWMB, 0);
  }
}

// function to stop both motors
void stopMotors() {
  analogWrite(PWMA, 0);
  analogWrite(PWMB, 0);
}

//reading the distance for the sensor
float readDistance() {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(3);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

```

```

    long duration = pulseIn(echoPin, HIGH, 25000); // timeout added
    if (duration == 0) return -1;                  // sensor failed
    return duration * 0.034 / 2.0;                 // convert to cm
}

//LED matrix code for printing the circle
void showCircle() {
    static uint8_t circle[8][12] = {
        { 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0 },
        { 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0 },
        { 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1 },
        { 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 },
        { 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 },
        { 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1 },
        { 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0 },
        { 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0 }
    };

    matrix.renderBitmap(circle, 8, 12);
}

// Clears the LED matrix when the obstacle is gone
void clearMatrix() {
    matrix.clear();
}

```

Logs

11/17 Meeting:

- Met in class to begin the final project
- Fully assembled the robot software, attached motors, wheels, and began WiFi connection
- Wired the motor driver
- Wired the sensor but did not code it yet
- Uploaded a basic motor test to confirm wiring, because we did not realize we needed a battery pack
- Code:

```

const int AIN1 = 13;
const int AIN2 = 12;
const int PWMA = 11;

```

```

const int PWMB = 10;
const int BIN2 = 9;
const int BIN1 = 8;

```

```

void setup() {
    pinMode(AIN1, OUTPUT);
}

```

```

pinMode(AIN2, OUTPUT);
pinMode(PWMA, OUTPUT);

pinMode(BIN1, OUTPUT);
pinMode(BIN2, OUTPUT);
pinMode(PWMB, OUTPUT);

// Test forward movement
digitalWrite(AIN1, HIGH);
digitalWrite(AIN2, LOW);
analogWrite(PWMA, 150);

digitalWrite(BIN1, HIGH);
digitalWrite(BIN2, LOW);
analogWrite(PWMB, 150);
}

void loop() {}

```

11/19 Meeting:

- Successfully connected the Arduino to WiFi using the WifiS3.h
- Created the first working webpage using Up, Down, Left, Right
- Integrated the ultrasonic sensor
- Implemented the first obstacle detection: stop robot and show LED matrix circle
- Code:

```

if (currentLine.indexOf("GET /UP") >= 0) {
    rightMotor(100);
    leftMotor(-100);
}

if (currentLine.indexOf("GET /DOWN") >= 0) {
    rightMotor(-100);
    leftMotor(100);
}

if (currentLine.indexOf("GET /LEFT") >= 0) {
    rightMotor(-175);
    leftMotor(75);
}

if (currentLine.indexOf("GET /RIGHT") >= 0) {
    rightMotor(-75);
    leftMotor(175);
}

```

- Sensor:

```

if (currentLine.indexOf("GET /UP") >= 0) {
  rightMotor(100);
  leftMotor(-100);
}

if (currentLine.indexOf("GET /DOWN") >= 0) {
  rightMotor(-100);
  leftMotor(100);
}

if (currentLine.indexOf("GET /LEFT") >= 0) {
  rightMotor(-175);
  leftMotor(75);
}

if (currentLine.indexOf("GET /RIGHT") >= 0) {
  rightMotor(-75);
  leftMotor(175);
}

```

11/24 Meeting:

- Cleaned and combined the full movement, WiFi, sensor, and LED matrix
- Finalized the obstacle detection (stop, show circle, resume movement when safe)
- Debugged the issue with the robot stopping and not moving again
- Fixed multiple issues with port detection and upload errors
- Redesigned the web control page to have more UI features
 - KSU colors
 - Hover glow
 - Rounded edges
 - Better spacing
- Successfully tested final movement and UI
- Code:

```

// STOP if obstacle is too close
distance = readDistance();

if (distance > 0 && distance < 15) {
  if (!blocked) {
    blocked = true;
  }
}

```

```

    stopMotors();
    showCircle();
    Serial.println("Obstacle detected → STOP");
  }
}

// CLEAR warning when safe
else if (distance > 20 && blocked) {
  blocked = false;
  clearMatrix();
  Serial.println("Obstacle cleared → READY");
}

```

Testing and Debugging

Throughout the development process, we encountered several challenges that required systematic testing and debugging. One of the first issues involved Genevieve’s motors moving in the wrong direction. When we pressed “Up,” the robot would move backward, and the turning controls were also inconsistent. We figured this out by lifting the robot and checking wheel rotation, then correcting the motor control functions. We also were forced to alter the power on the motors when they drive due to faulty motors so we had to slightly adjust the power on the right motor to keep it in check with the other.

Another challenge was getting Genevieve to connect to the computer and the WiFi. For several attempts, no serial port appeared except for Bluetooth, which caused upload failures. By resetting the board, restarting the IDE, and resetting the computer we eventually got the right port. For the WiFi, we decided to connect via HotSpot, and that had to be turned off and on several times before it worked correctly.

Finally, we encountered most of our problems with the sensor and Genevieve moving after an obstacle was removed. The sensor initially returned inaccurate readings (0 or -1). Through serial monitoring and checking wiring photos, we discovered that the trig and echo wires were connected to the wrong pins, and fixing that fixed the distance measurement. Once the obstacle logic was working, the next challenge was that Genevieve would not resume movement

after an obstacle was removed. After testing many different approaches, we figured out that the behavior was correct for a manual control system: once the robot stopped for safety, it required a user to send a new command. We updated the code accordingly to support this design.

IP Page Explanation

For the IP control page, we designed the layout to use a game-style directional button arrangement so the robot feels easy to control. We chose Kent State colors, blue and gold, to keep the aesthetic consistent and visually appealing. The buttons are large, rounded, and spaced to make them easy to tap on any device. This design avoids any emojis or special characters to ensure compatibility with the Arduino's basic web server. Overall, the IP page is clean, readable, and functional, making the robot control very straightforward.

Team Reflection

1. What was the most challenging part?
 - a. The most challenging part of this project was figuring out how to code it so that after an obstacle is detected and removed the robot will resume movement. We explored multiple different logic structures, tested different versions of the code, and tried both autonomous and manual controls before we found something that worked.
2. How did you divide responsibilities and communicate?
 - a. We discussed what each person's strengths were, and went from there. It was much easier to divide work since we both excelled in different areas of the project. We communicated mostly in class and via email. One of us focused more

on the hardware assembly, wiring, and TinkerCAD, while the other focused on coding the robot functions, WiFi control, and HTML.

3. What did you learn about debugging code and circuits?
 - a. We learned that debugging is an iterative, step-by-step process that requires a lot of patience, testing, and checking every detail. Small mistakes, such as a reversed motor wire or wrong serial port, can cause major behavior issues. We also learned the importance of using Serial Monitor to print the values continuously, especially when diagnosing ultrasonic sensor readings and movement logic. Debugging taught us how important it is to verify the hardware first, then fix the software issues one at a time.
4. If you had more time, what feature would you add?
 - a. If we had more time, we would add autonomous navigation so Genevieve could fully avoid obstacles on her own without relying on manual input. That way we could just press one direction one time and it could move itself around all obstacles.
 - b. We would also love to add more sensors in order to have better detection from all around the robot so she can fully move around and detect her surroundings.
 - c. Finally we would love to add a speedometer that we can manually change. Being able to speed up and slow down Genevieve would really enhance her movement capabilities.
5. How could this project connect to a real-world application?
 - a. This project mirrors real-world systems that use remote control and obstacle detection such as delivery robots, smart vacuum cleaners, and warehouse robots.

The combination of WiFi control and safety based sensor logic is directly applicable to real robotics applications where both user commands and automated safety behavior must work together.

Demo

<https://youtube.com/shorts/-L8QJmt0IfI?feature=share>